

SDE INTERN ASSESSMENT

Automated Job Application Dashboard

Job Scraping • Greenhouse Form Automation • Headless Browser • Resume Upload • Screenshot Proof

1. Assessment Overview

The candidate is required to build a full-stack Job Application Automation Dashboard. The system must collect approximately 10–15 job openings from a publicly accessible company career page that uses Greenhouse, display those jobs in a dashboard, and provide automated application workflows.

For each selected job, the system should use headless browser automation to open the application page, populate the available application fields using a stored dummy candidate profile, upload a resume, and stop at the final review/submission stage. The system must capture a screenshot at that stage and associate the proof with the relevant job card.

2. Objective

The assessment is designed to evaluate the candidate's ability to build an end-to-end software system involving:

- Web scraping and structured job-data extraction
- Frontend dashboard development
- Backend API and data management
- Headless browser automation
- Dynamic form detection and form filling
- File upload handling
- Workflow/state management
- Screenshot generation and storage
- Error handling and retry logic
- Clean code, architecture, documentation, and engineering practices

3. Core Problem Statement

Build a dashboard that contains 10–15 jobs collected from a Greenhouse-based career page. The user should be able to apply to an individual job or initiate the application workflow for all jobs. The automation must complete the application form as far as reasonably possible, upload the candidate's resume, reach the final review/submit stage, capture evidence, and stop before the actual application is submitted.

4. Functional Requirements

4.1 Job Collection

- Select a publicly accessible company career page using Greenhouse.
- Collect approximately 10–15 job openings.
- Do not manually hardcode the complete job list.
- Store the collected jobs in a structured data store.
- Prevent duplicate jobs where practical.

Minimum job information:

Field	Description
Job ID	Unique identifier for the job
Job Title	Title of the position
Company	Company name
Location	Job location
Job Description	Relevant description or summary
Job URL	Public job-detail URL
Application URL	Greenhouse application URL
Source	Career page/source
Status	Current automation/application state
Screenshot	Path or reference to captured proof

4.2 Dashboard

The dashboard must display the collected jobs as individual cards or equivalent list items.

- Search jobs by title or relevant text.
- Display company, location, and job information.
- Provide a View Job option.
- Provide an Apply option for each individual job.
- Display the current application automation status.
- Display or preview the screenshot after successful capture.
- Provide an Apply to All action.

4.3 Candidate Profile

Candidate information must be stored separately from the application logic, preferably in a local JSON/YAML file or an equivalent structured data source.

Field	Example
First Name	Rahul
Last Name	Sharma
Email	rahul.sharma@example.com
Phone	+91 9000000000
Location	Bangalore
LinkedIn	https://linkedin.com/in/example
GitHub	https://github.com/example
Education	B.E. Computer Science

Experience	Fresher
Skills	JavaScript, React, Node.js, Python
Resume	./data/resume.pdf

4.4 Automated Application Workflow

For an individual job, the automation should:

1. Launch a headless browser using an automation framework such as Playwright, Puppeteer, or Selenium.
2. Navigate to the Greenhouse application page.
3. Identify and fill supported candidate fields.
4. Upload the candidate resume.
5. Fill relevant additional application questions when the answer can be safely determined from the candidate profile or predefined dummy data.
6. Handle ordinary multi-step application pages where possible.
7. Detect the final review/submission stage.
8. Capture a screenshot of the final stage.
9. Store the screenshot and update the job's status.
10. Stop before the actual submission action.

4.5 Apply to All

The dashboard must provide an Apply to All option. The system should process jobs in a controlled manner, preferably sequentially unless the candidate has a well-designed concurrency strategy.

Example status progression:

- NOT_STARTED
- PROCESSING
- FORM_FILLED
- READY_FOR_SUBMISSION
- SCREENSHOT_CAPTURED
- FAILED

4.6 Screenshot Proof

After the automation reaches the final review/submission stage, it must capture a screenshot that clearly demonstrates the application has been filled and is ready for submission.

Example storage structure:

- screenshots/job_001.png
- screenshots/job_002.png
- screenshots/job_003.png

The corresponding screenshot reference should be linked to the relevant job record and made accessible from its dashboard card.

5. Important Safety and Assessment Constraint

The candidate must NOT submit any real job application.

The automation must stop before the final submission action. The purpose is to demonstrate reliable automation and proof of reaching the final stage without sending applications to real employers.

- Do not click or trigger the final Submit Application action.
- Do not bypass CAPTCHA, MFA, bot protection, or other security mechanisms.
- If a CAPTCHA or unsupported security challenge appears, mark the job as requiring manual intervention or failed and capture appropriate evidence if possible.
- Use dummy candidate information and a dummy/test resume for the assessment.

6. Error Handling

The application should handle common failures gracefully, including:

- Job no longer available
- Application page timeout
- Unexpected page navigation
- Missing or unsupported form field
- Resume upload failure
- Required question that cannot be answered automatically
- Browser/automation failure
- CAPTCHA or other anti-bot challenge

A failed job should not stop the entire Apply to All process. The system should record the failure reason and continue with the remaining jobs where appropriate.

7. Suggested Technical Architecture

The technology stack is intentionally flexible. Candidates may use technologies they are comfortable with, provided the final system satisfies the requirements.

Layer	Suggested Technologies
Frontend	React / Next.js / equivalent
Backend	Node.js + Express / Next.js API / equivalent
Database	SQLite / PostgreSQL / MongoDB / structured local storage
Automation	Playwright / Puppeteer / Selenium
File Storage	Local filesystem or equivalent
Language	JavaScript / TypeScript / Python or equivalent

8. Suggested API Design

The following API structure is optional and is provided as guidance, not a strict requirement.

Method	Endpoint	Purpose
GET	/api/jobs	Retrieve collected jobs
GET	/api/jobs/:id	Retrieve one job
POST	/api/jobs/scrape	Start/update job collection
POST	/api/applications/:jobId/apply	Start automation for one job
POST	/api/applications/apply-all	Start controlled processing for all jobs
GET	/api/applications/:jobId/status	Get automation status
GET	/api/applications/:jobId/screenshot	Access captured screenshot
GET	/api/candidate	Retrieve candidate profile

9. UI Expectations

The UI does not need to be visually complex. It should be clean, readable, responsive, and easy to operate.

A job card should ideally contain:

- Job title
- Company
- Location
- Short job description
- Application status
- Apply button
- View Job button
- Screenshot preview/view action when available
- Failure reason when applicable

10. Apply-to-All Experience

When Apply to All is selected, the dashboard should clearly communicate progress.

Job	Status	Evidence
Software Engineer Intern	Ready for Submission	Screenshot Available
Frontend Developer Intern	Screenshot Captured	Screenshot Available
Backend Engineer Intern	Processing	Pending
Data Analyst Intern	Failed	Failure Reason
ML Intern	Not Started	Pending

11. Assessment Scoring

Assessment Area	Weight
Job scraping and data extraction	15
Dashboard and frontend usability	10
Backend/API design	10
Greenhouse application automation	20
Candidate data and resume handling	10

Apply-to-All workflow	10
Screenshot/proof mechanism	10
Error handling and resilience	5
Code quality and architecture	5
Documentation and setup instructions	5
Total	100

Recommended passing score: 60/100.

12. Bonus Points

- Robust handling of multi-step application forms
- Reusable form-field mapping logic
- Retry mechanism with sensible limits
- Application queue with progress tracking
- Detailed automation logs
- Strong validation and type safety
- Unit/integration tests
- Docker setup
- Well-designed database schema
- Clean responsive UI

13. Deliverables

- Complete source code repository
- Working frontend dashboard
- Working backend/API where applicable
- Job scraper/collector
- Headless application automation
- Dummy candidate profile file
- Dummy resume file
- Captured screenshots demonstrating final-stage application state
- README with setup and execution instructions
- Environment-variable example file if configuration is required

14. README Requirements

The repository README should explain:

- Project overview
- Technology stack
- Prerequisites
- Installation steps
- How to start the application
- How to run the scraper

- How to configure candidate data
- How to run individual application automation
- How to run Apply to All
- Where screenshots are stored
- Known limitations
- Important note that actual applications are never submitted

15. Suggested Project Structure

job-automation-dashboard/

```
├── frontend/
├── backend/
├── scraper/
├── automation/
├── data/
|   ├── candidate.json
|   └── resume.pdf
├── screenshots/
├── tests/
├── .env.example
└── README.md
```

16. Evaluation Focus

The assessment is not primarily a UI-design test. Reviewers should focus on whether the candidate can build a reliable end-to-end system and make sensible engineering decisions.

- Does the candidate understand dynamic web pages and structured data extraction?
- Is the automation reliable and maintainable?
- Can the system process multiple jobs without losing state?
- Does the candidate handle failures instead of hiding them?
- Is the code modular and reusable?
- Can another developer set up and run the project from the README?
- Does the implementation stop safely before real submission?

17. Final Expected Outcome

At the end of the assessment, the reviewer should be able to open the dashboard, see approximately 10–15 scraped jobs, start an individual application or Apply to All workflow, observe automated form filling and resume upload, and see a screenshot associated with each successfully processed job showing that the automation reached the final submission stage.

18. Candidate Submission Checklist

- 10–15 jobs successfully collected
- Dashboard displays the jobs
- Individual Apply workflow works
- Apply to All workflow works
- Candidate profile is loaded from structured data
- Resume is uploaded automatically
- Automation reaches final review/submission stage
- Actual submission is never performed
- Screenshot is captured
- Screenshot is visible against the corresponding job
- Failure states are recorded
- README is complete